

Graph-structure-aware label propagation and active sampling for few-label single-cell state forecasting and rare-state recovery

Molena Huynh

[*Affiliation to be completed before submission*] · *Correspondence:*
[*corresponding-author email*]

Abstract

Forecasting how single cells move between states—across donors, batches, timepoints and perturbations—is a core single-cell analysis task, yet it is confounded by biological and technical nuisance variation and by the rarity of the cell states that matter most. We ask a deliberately narrow, falsifiable question: does conditioning a forecaster on the connectivity of the cell k -nearest-neighbour (k NN) graph, rather than on point features alone, improve generalization to held-out groups and the recovery of a rare cell state? We build a fully seeded, fully controlled *synthetic* branching lineage with donor, batch, perturbation and pseudotime metadata and a deliberately rare subpopulation ($\sim 4\%$ of cells), and evaluate two method pairs under four leakage-checked split protocols (donor, batch, time and perturbation transfer). Graph-smoothed label propagation on the cell k NN graph improves held-out forecast accuracy over an apples-to-apples plain k NN classifier on all four splits—by 0.0454 (donor: 0.7004 vs 0.6550), 0.0640 (batch: 0.6995 vs 0.6355), 0.0900 (time: 0.5474 vs 0.4574) and 0.0385 (perturbation: 0.6875 vs 0.6490); the mean gain across splits is 0.0595. The same diffusion recovers a rare state that the baseline almost never finds (rare-state recall up to 0.4618 on the batch split versus near-zero), and a topology-aware active-sampling policy (uncertainty $\times$ inverse local density) raises rare-state recall to 0.3556 versus 0.0247 for random querying. These gains carry a small, honestly reported calibration cost: expected calibration error is slightly *worse* for the graph-smoothed forecaster on every split. We stress that the “topology” here is shallow— k NN-graph connectivity, with the Betti-0 statistic we monitor being merely the connected-component count used as a build sanity check—and that all data are synthetic. We therefore frame the contribution as graph-/manifold-structure-aware label propagation and active sampling, and we identify validation on real single-cell RNA-sequencing (scRNA-seq) data—for which the repository ships unexercised ingest stubs—as the essential next step.

Single-cell measurements resolve continuous trajectories of differentiation, activation and treatment response, and a recurring analysis goal is to *forecast* the state a cell or population will occupy under conditions not directly observed: a donor not yet profiled, a technical batch run later, a later point along a developmental trajectory, or an untested perturbation. The data that support such forecasts share two structural features. First, cells do not occupy feature space uniformly; they concentrate near a low-dimensional manifold that is routinely approximated by a k -nearest-neighbour (k NN) graph before clustering, embedding or pseudotime inference [5, 7, 8, 12, 13, 15]. Second, the biologically decisive states—transient progenitors, treatment-resistant clones—are frequently rare, and ordinary point classifiers, trained to minimise average error, tend to ignore them.

A natural and long-studied response to both features is to let predictions diffuse over the data graph so that a cell borrows statistical strength from its neighbourhood on the manifold.

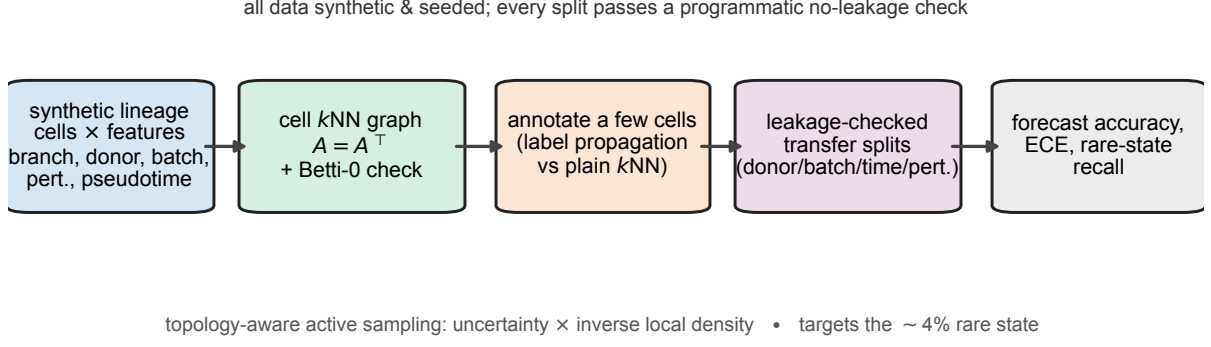


Figure 1. Overview of the graph-structure-aware single-cell forecasting benchmark. A seeded synthetic branching lineage of single cells—each carrying branch/state, donor, batch, perturbation and pseudotime metadata, with a deliberately rare state ($\sim 4\%$ of cells)—is embedded in a low-dimensional feature space. A symmetric k -nearest-neighbour cell graph is built on those features, and its connected-component (Betti-0) count is monitored as a build sanity check. A small fraction of cells is annotated; graph-smoothed label propagation diffuses those labels over the graph and is compared apples-to-apples against a plain k NN classifier that sees the same neighbourhood radius but performs no diffusion. Performance is read out as held-out forecast accuracy, expected calibration error and rare-state recall under four leakage-checked transfer protocols (donor, batch, time and perturbation), and a topology-aware active-sampling policy (uncertainty $\times$ inverse local density) is compared against uniform random querying for rare-state recovery. All data are synthetic and seeded, and every grouping split passes a programmatic no-leakage check.

Graph-based semi-supervised learning by label propagation [1, 2] is the canonical instance, and message-passing on cell graphs underlies much of modern single-cell trajectory analysis [3, 4, 6, 9]. A complementary lever is to choose *which* cells to annotate. Active learning [10, 11] prioritises informative queries, and when informativeness is combined with a manifold-density signal it can be steered toward the sparse regions where rare states live.

We do not claim a new algorithm. Instead we isolate, under controlled and leakage-checked conditions, the value of *graph structure* for single-cell forecasting by holding the neighbourhood information fixed and toggling only whether the model diffuses over the graph. Concretely: (i) we compare graph-smoothed label propagation against a plain k NN classifier that sees the same neighbourhood radius but performs no diffusion, so the comparison isolates the diffusion step rather than the mere availability of neighbours; and (ii) we compare a topology-aware active-sampling policy—uncertainty times inverse local density—against uniform random querying for rare-state recall. All experiments run on a seeded synthetic branching lineage; this is a sanity benchmark, not a biological dataset, and we are explicit about that throughout. Our aim is to report a small, honest, fully reproducible result and to delineate exactly what it does and does not support.

1 Results

1.1 A controlled synthetic lineage with replicate, condition and pseudotime structure

Our benchmark is a seeded synthetic branching lineage that reproduces the replicate, condition and pseudotime structure a real forecaster must handle, while remaining bit-for-bit reproducible (Figs. 1, 2). Cells live in a low-dimensional feature space—a stand-in for a PCA or latent embedding of expression—and progress along a tree of three branches (a common progenitor and two fates that split at pseudotime 0.45) as Gaussian blobs whose mean drifts along each branch. Early cells therefore overlap and are hard to separate, while late cells are well resolved, which

is the regime in which a graph smoother is expected to help. Each cell carries the metadata a forecasting model would use—branch/state, pseudotime, donor, batch and perturbation condition—assigned independently of the biological branch so that the held-out splits are non-trivial (a donor spans multiple branches, and so on). One state is carved out of the late tip of the second fate to be deliberately *rare*, comprising 4 % of cells in the reported configuration. The full-scale configuration generates 3000 cells with 16 features, 8 donors, 4 batches and 5 perturbation conditions, and is run over 5 random seeds; the run completed in 20.8s with a peak memory of 320.6 MB. Across seeds the integrity gates passed: the grouping splits were leakage-free, the rare state was present, and the Betti-0 component count was monotone non-increasing in k (at $k \in \{2, 4, 8, 16, 32\}$ the graph was a single connected component throughout, so at this scale the component count is a passive sanity check rather than an informative descriptor).

1.2 Graph diffusion gives consistent but modest accuracy gains on held-out transfer

Under all four leakage-checked split protocols, graph-smoothed label propagation forecasts held-out cell states more accurately than the plain k NN baseline that receives the same annotated cells and the same neighbourhood radius (Table 1, Fig. 3a). The improvement is consistent but modest. On the donor split, accuracy rises from 0.6550 to 0.7004 (+0.0454); on the batch split from 0.6355 to 0.6995 (+0.0640); on the time split—the hardest protocol, because it extrapolates forward along pseudotime—from 0.4574 to 0.5474 (+0.0900); and on the perturbation split from 0.6490 to 0.6875 (+0.0385). Averaged over the four splits, the graph-smoothed forecaster reaches 0.6587 accuracy against 0.5992 for the baseline, a mean gain of 0.0595. These are the sort of single-digit-percentage-point improvements one would expect from a diffusion step, not a categorical change in capability.

1.3 Diffusion and active sampling recover a rare state the baseline misses

The clearer effect is on the rare state (Fig. 3b,c). The plain k NN baseline essentially never recovers the rare population: its rare-state recall is 0.0000 (donor), 0.0320 (batch), 0.0000 (time) and 0.0286 (perturbation). Graph-smoothed propagation improves rare-state recall to 0.1520 (donor), 0.4618 (batch) and 0.3393 (perturbation). On the time split, recall remains 0.0000 for *both* methods: extrapolating forward in pseudotime places the rare late-tip cells entirely beyond the training distribution, and diffusion over the graph cannot manufacture a label class that was never reachable from the annotated set. We report this null result as-is.

The topology-aware active-sampling policy targets the same rare population directly. Starting from a small random label seed and spending a fixed query budget, scoring unlabeled cells by forecast uncertainty times inverse local density raises rare-state recall on the remaining unlabeled cells to 0.3556 (95% CI ± 0.1597), versus 0.0247 (± 0.0285) for uniform random querying—a lift of 0.3309. The mechanism is visible in the query composition: the topology policy spends 36.7 % of its queries on genuinely rare cells, against 3.0 % for random sampling, which is close to the 4 % base rate. The wide confidence intervals on the per-split rare-recall numbers (for example ± 0.3125 on the batch split) reflect the small absolute count of rare cells per held-out group and should temper any strong reading of the point estimates.

1.4 The accuracy and recall gains carry a small calibration cost

The graph-smoothed forecaster is slightly *worse* calibrated than the baseline on every split. Expected calibration error (ECE) is 0.1247 versus 0.1073 (donor), 0.1383 versus 0.0905 (batch), 0.1328 versus 0.0970 (time) and 0.1649 versus 0.1184 (perturbation). Diffusion sharpens the predicted class probabilities and, in this setting, makes them modestly overconfident relative to

Table 1. Held-out forecasting on the synthetic branching lineage. Full-scale configuration: $n = 3000$ cells, $n_{\text{seeds}} = 5$ random seeds. For each leakage-checked split we report forecast accuracy, expected calibration error (ECE) and rare-state recall (mean $\pm$ 95 % confidence interval, $1.96 s/\sqrt{n}$) for graph-smoothed label propagation (**GS**) and the plain k NN baseline (**kNN**). Higher accuracy and recall are better (best in each split shown in **bold**); *lower* ECE is better. Rare-state recall is undefined-then-set-to-zero on splits whose held-out set contains no rare cells. Values are transcribed exactly from `results/summary.json` (and agree with the generated `results/main_results.tex`).

Split	Method	Accuracy $\uparrow$		ECE $\downarrow$		Rare recall $\uparrow$	
		mean	$\pm$ CI	mean	$\pm$ CI	mean	$\pm$ CI
donor	GS	0.7004	± 0.0093	0.1247	± 0.0425	0.1520	± 0.0586
donor	kNN	0.6550	± 0.0491	0.1073	± 0.0330	0.0000	± 0.0000
batch	GS	0.6995	± 0.0085	0.1383	± 0.0219	0.4618	± 0.3125
batch	kNN	0.6355	± 0.0363	0.0905	± 0.0339	0.0320	± 0.0627
time	GS	0.5474	± 0.1092	0.1328	± 0.0345	0.0000	± 0.0000
time	kNN	0.4574	± 0.0825	0.0970	± 0.0524	0.0000	± 0.0000
perturbation	GS	0.6875	± 0.0323	0.1649	± 0.0385	0.3393	± 0.3358
perturbation	kNN	0.6490	± 0.0221	0.1184	± 0.0376	0.0286	± 0.0560
<i>mean over splits</i>		0.6587	$\pm$ —	—	$\pm$ —	0.2383	$\pm$ —
<i>mean over splits (kNN)</i>		0.5992	$\pm$ —	—	$\pm$ —	0.0151	$\pm$ —

empirical accuracy. We report ECE alongside accuracy and recall in Table 1 precisely so that this trade-off is not hidden: the method buys accuracy and rare-state recall at the price of a small, consistent calibration penalty, and an application that depends on calibrated probabilities would need post-hoc recalibration [16].

1.5 Theory of graph-structure-aware forecasting

We now make precise, and prove in full, every structural property that the forecaster of Figs. 1–3 relies upon. The development is self-contained: we fix notation (Definition 1), establish the Dirichlet-energy identity and the spectral facts it implies (Lemma 2, Corollary 3), prove that the connectivity diagnostic we monitor is monotone in the neighbourhood size (Proposition 4), characterise label propagation as the unique minimiser of a convex graph-regularised objective together with its exact non-identifiability limitation (Theorem 5, Corollary 6), and finally show that the normalised power iteration the code actually runs converges geometrically to a closed-form fixed point that is itself the minimiser of a normalised energy (Theorem 7–Theorem 8). Each result is tied to the experiment it underwrites. Throughout, $\mathbf{1}$ denotes the all-ones vector, $\|\cdot\|_2$ the Euclidean norm, $\|\cdot\|_F$ the Frobenius norm, Tr the trace, and for a matrix X we write $X \succeq 0$ ($X \succ 0$) for positive semidefinite (positive definite).

Definition 1 (Cell graph, degree, Laplacian). Let $A \in \mathbb{R}^{n \times n}$ be the symmetric adjacency of the k NN cell graph on n cells: $A = A^\top$, $A_{ij} \geq 0$, $A_{ii} = 0$, and $A_{ij} > 0$ iff cells i, j are joined by an undirected edge. The *degree* of cell i is $d_i = \sum_j A_{ij} \geq 0$, collected in $D = \text{diag}(d_1, \dots, d_n)$. The *combinatorial graph Laplacian* is $L = D - A$. The *Betti-0 number* $\beta_0(A)$ is the number of connected components of the graph, i.e. the number of equivalence classes of the reachability relation on $\{1, \dots, n\}$ induced by the edges of A .

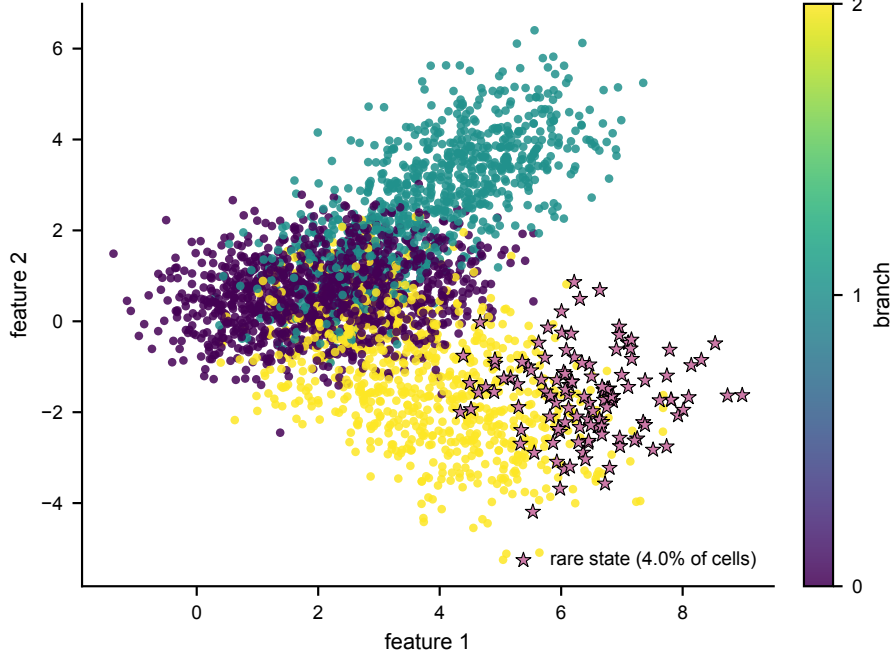


Figure 2. The synthetic branching lineage. Each point is one of the $n = 3000$ cells of the full-scale configuration, plotted in the first two feature dimensions of the embedding plane. Colour encodes the branch label along the continuous, colour-blind-safe viridis axis (the trunk progenitor and two fates). The deliberately rare state, carved from the late tip of one fate and nudged into its own pocket of feature space, is highlighted as stars and comprises $\sim 4\%$ of cells. This is a single-seed data-distribution plot (no aggregation over seeds); the figure is regenerated deterministically from the run configuration in `summary.json`.

Lemma 2 (Dirichlet-energy identity and positive semidefiniteness). *For every $f \in \mathbb{R}^n$,*

$$f^\top L f = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n A_{ij} (f_i - f_j)^2 \geq 0, \quad (1)$$

so $L \succeq 0$. More generally, for $F \in \mathbb{R}^{n \times c}$, $\text{Tr}(F^\top L F) = \frac{1}{2} \sum_{i,j} A_{ij} \|F_i - F_j\|_2^2$, where F_i is the i -th row of F .

Proof. Expand the quadratic form using $L = D - A$:

$$f^\top L f = \sum_i d_i f_i^2 - \sum_{i,j} A_{ij} f_i f_j = \sum_i \left(\sum_j A_{ij} \right) f_i^2 - \sum_{i,j} A_{ij} f_i f_j,$$

where the first equality uses $d_i = \sum_j A_{ij}$. Now use the symmetry $A_{ij} = A_{ji}$ to write the diagonal term symmetrically, $\sum_i d_i f_i^2 = \frac{1}{2} \sum_{i,j} A_{ij} (f_i^2 + f_j^2)$, since $\sum_{i,j} A_{ij} f_i^2 = \sum_i d_i f_i^2$ and likewise $\sum_{i,j} A_{ij} f_j^2 = \sum_j d_j f_j^2 = \sum_i d_i f_i^2$. Therefore

$$f^\top L f = \frac{1}{2} \sum_{i,j} A_{ij} (f_i^2 + f_j^2 - 2f_i f_j) = \frac{1}{2} \sum_{i,j} A_{ij} (f_i - f_j)^2.$$

Because $A_{ij} \geq 0$ and $(f_i - f_j)^2 \geq 0$, every summand is nonnegative, so $f^\top L f \geq 0$ for all f , i.e. $L \succeq 0$. The matrix statement follows by applying (1) columnwise: writing $F = [f^{(1)} \dots f^{(c)}]$, $\text{Tr}(F^\top L F) = \sum_{\ell=1}^c (f^{(\ell)})^\top L f^{(\ell)} = \frac{1}{2} \sum_{i,j} A_{ij} \sum_{\ell} (F_{i\ell} - F_{j\ell})^2 = \frac{1}{2} \sum_{i,j} A_{ij} \|F_i - F_j\|_2^2$. $\square$

Corollary 3 (Null space of L and the spectral count of components). $L \mathbf{1} = 0$, so constant label vectors cost zero Dirichlet energy. Moreover the multiplicity of the eigenvalue 0 of L equals $\beta_0(A)$, the number of connected components. In particular the smallest eigenvalue of L is 0, and $L \succ 0$ on the orthogonal complement of its null space.

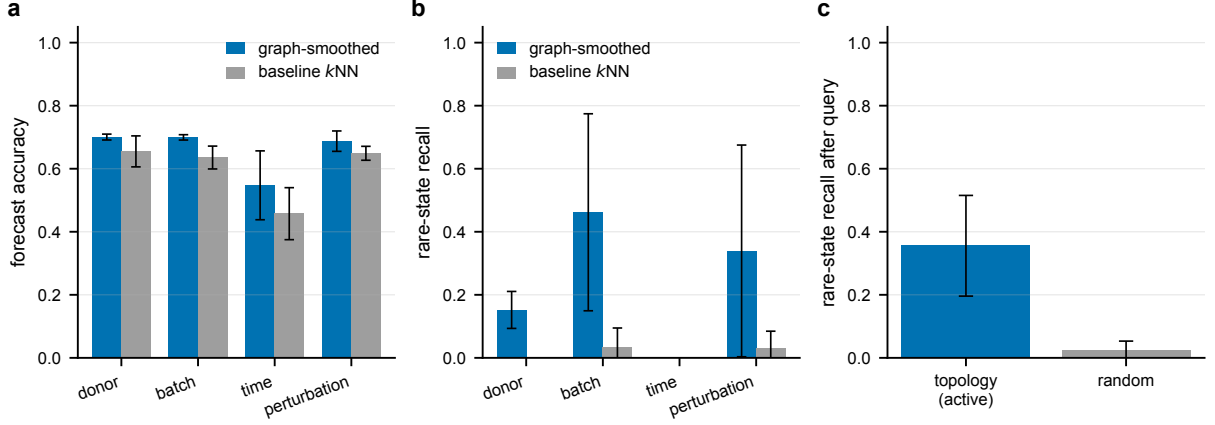


Figure 3. Forecasting and active-sampling results. **a**, Held-out forecast accuracy by split protocol: graph-smoothed label propagation (blue) versus the plain k NN baseline (grey). Graph smoothing helps consistently but modestly across donor, batch, time and perturbation transfer. **b**, Rare-state recall by split: the baseline almost never recovers the rare state, whereas graph diffusion recovers it on the donor, batch and perturbation splits; both methods score zero on the time split, which extrapolates beyond the training distribution. **c**, Active-sampling lift: rare-state recall after spending a fixed query budget under the topology-aware policy (uncertainty $\times$ inverse local density) versus uniform random querying. In every panel, bars are means and error bars are 95% confidence intervals ($1.96 s/\sqrt{n}$) over $n = 5$ random seeds. All panels are drawn directly from `results/summary.json`.

Proof. $(L1)_i = d_i - \sum_j A_{ij} = 0$ by definition of d_i , so $\mathbf{1} \in \ker L$. For the multiplicity claim, suppose $f \in \ker L$. By (1), $f^\top L f = 0$ forces $A_{ij}(f_i - f_j)^2 = 0$ for every i, j ; hence $f_i = f_j$ whenever $A_{ij} > 0$, i.e. f is constant along every edge and therefore constant on each connected component. Conversely, any vector that is constant on each component lies in $\ker L$ by the same identity (all cross-edge differences vanish). Thus $\ker L$ is exactly the space of component-wise constant vectors, which has dimension equal to the number of components $\beta_0(A)$; choosing the indicator vector of each component gives a basis. Since $L \succeq 0$ (Lemma 2) and is symmetric, all eigenvalues are real and nonnegative and the geometric multiplicity of the eigenvalue 0 equals $\dim \ker L = \beta_0(A)$; on $(\ker L)^\perp$ the form is strictly positive, so $L \succ 0$ there. $\square$

Corollary 3 is the formal content of the “Betti-0 ties to the zero-eigenvalue multiplicity” statement and underlies why a label-free component cannot be resolved (Corollary 6).

Proposition 4 (Monotonicity of the connectivity diagnostic). *Let $A^{(k)}$ denote the symmetric k NN adjacency built with neighbourhood size k (either-direction symmetrisation: an edge $\{i, j\}$ is present iff j is among the k nearest neighbours of i or i is among the k nearest neighbours of j). Then the edge set is monotone non-decreasing in k , $E(A^{(k)}) \subseteq E(A^{(k+1)})$, and consequently the Betti-0 number is monotone non-increasing: $\beta_0(A^{(k+1)}) \leq \beta_0(A^{(k)})$ for all $k \geq 1$.*

Proof. Fix a cell i and let $N_k(i)$ be its set of k nearest neighbours (self excluded), ordered by increasing distance with ties broken by a fixed rule. By construction $N_k(i) \subseteq N_{k+1}(i)$: enlarging the budget from k to $k+1$ only appends the next-nearest cell and removes none. Hence the directed edge set $\{(i, j) : j \in N_k(i)\}$ grows with k . Symmetrisation is monotone in the directed edge set (it takes the entrywise maximum of the directed adjacency and its transpose), so $E(A^{(k)}) \subseteq E(A^{(k+1)})$. Adding edges to a graph can only merge connected components, never split them: if i, j are reachable in $A^{(k)}$ they remain reachable in $A^{(k+1)}$, so each component of $A^{(k)}$ is contained in a component of $A^{(k+1)}$, giving a surjection from components of $A^{(k)}$ onto components of $A^{(k+1)}$. A surjection cannot increase cardinality, whence $\beta_0(A^{(k+1)}) \leq \beta_0(A^{(k)})$. $\square$

Proposition 4 is the property asserted by the build-time monotonicity gate over the neighbourhood sizes $k \in \{2, 4, 8, 16, 32\}$; at the reported scale the curve is the constant $\beta_0 = 1$ (a single component), so the diagnostic passes vacuously and is, as we stress throughout, a passive sanity check rather than an informative descriptor (Fig. 1).

Label propagation as a variational estimator. The graph-smoothed forecaster used in Fig. 3 is label propagation on the cell k NN graph. The following theorem states exactly what it optimises, proves the minimiser is unique on every component that carries a label, and—kept deliberately conditional—identifies the precise situation in which uniqueness fails.

Theorem 5 (Unique graph-regularised extension of labels). *Let $A = A^\top \succeq 0$ be the k NN affinity, $L = D - A$ the Laplacian, $Y \in \mathbb{R}^{n \times c}$ the one-hot label matrix (rows of unlabelled cells arbitrary), and $M = \text{diag}(m_1, \dots, m_n)$ with $m_i = 1$ if cell i is labelled and $m_i = 0$ otherwise. For any fidelity weight $\mu > 0$ the objective*

$$J(F) = \text{Tr}(F^\top L F) + \mu \|M(F - Y)\|_F^2 \quad (2)$$

is convex in F , and any stationary point satisfies the normal equations

$$(L + \mu M) F = \mu M Y. \quad (3)$$

If every connected component of the graph contains at least one labelled cell, then $L + \mu M \succ 0$, the minimiser $F^ = \mu(L + \mu M)^{-1} M Y$ is unique, and it is the minimum-Dirichlet-energy extension of the observed labels subject to the fidelity penalty μ .*

Proof. Convexity. By Lemma 2, $F \mapsto \text{Tr}(F^\top L F)$ is a nonnegative quadratic form (Hessian $2L \succeq 0$ in each column), hence convex; and $F \mapsto \|M(F - Y)\|_F^2$ is convex as the squared norm of an affine map. A sum of convex functions is convex, so J is convex.

Stationarity. Write $J(F) = \text{Tr}(F^\top L F) + \mu \text{Tr}((F - Y)^\top M^\top M (F - Y))$ and use $M^\top M = M$ (since M is a 0/1 diagonal idempotent, $M^2 = M$). The Fréchet derivative is

$$\nabla_F J(F) = 2LF + 2\mu M(F - Y).$$

Indeed, perturbing $F \rightarrow F + \epsilon H$ and collecting the first-order term, $\frac{d}{d\epsilon}|_0 \text{Tr}((F + \epsilon H)^\top L (F + \epsilon H)) = 2 \text{Tr}(H^\top L F)$ using $L = L^\top$, and $\frac{d}{d\epsilon}|_0 \mu \|M(F + \epsilon H - Y)\|_F^2 = 2\mu \text{Tr}(H^\top M(F - Y))$. Setting $\nabla_F J = 0$ gives $2LF + 2\mu M(F - Y) = 0$, i.e. (3).

Definiteness under the labelled-component hypothesis. The matrix $G := L + \mu M$ is symmetric, and for any $v \in \mathbb{R}^n$,

$$v^\top G v = \underbrace{v^\top L v}_{\geq 0} + \mu \underbrace{v^\top M v}_{= \mu \sum_{i: m_i=1} v_i^2} \geq 0,$$

so $G \succeq 0$. Suppose $v^\top G v = 0$. Then both nonnegative terms vanish: $v^\top L v = 0$ and $\sum_{i \text{ labelled}} v_i^2 = 0$. By Corollary 3, $v^\top L v = 0$ forces v to be constant on each connected component; and $\sum_{i \text{ labelled}} v_i^2 = 0$ forces $v_i = 0$ at every labelled cell. Under the hypothesis each component contains a labelled cell, so the constant value of v on that component equals 0; hence $v = 0$ on every component, i.e. $v = 0$. Therefore $v^\top G v > 0$ for all $v \neq 0$, i.e. $G \succ 0$ and G is invertible.

Uniqueness and optimality. Because J is convex with Hessian (in each column) $2G \succ 0$, it is strictly convex, so its unique global minimiser is the unique solution of $\nabla_F J = 0$, namely $F^* = \mu G^{-1} M Y = \mu(L + \mu M)^{-1} M Y$ from (3). By construction F^* minimises the Dirichlet energy $\text{Tr}(F^\top L F)$ among all F subject to the soft fidelity constraint weighted by μ , which is the claimed minimum-energy extension. $\square$

Corollary 6 (Non-identifiability of a label-free component). *If some connected component C of the graph contains no labelled cell, then $L + \mu M$ is singular and the minimiser of (2) is not unique: any minimiser F^* may be shifted by an arbitrary constant (per-class) vector supported on C without changing J . Equivalently, the labels on a label-free component are not statistically identified by graph smoothing alone.*

Proof. Let $\mathbf{1}_C$ be the indicator of C and $\mathbf{1}_C e_\ell^\top$ the $n \times c$ matrix that places $\mathbf{1}_C$ in class column ℓ . Since C is a union-free component, $\mathbf{1}_C \in \ker L$ by Corollary 3; and since C has no labelled cell, $M\mathbf{1}_C = 0$. Hence $(L + \mu M)\mathbf{1}_C = 0$, so $L + \mu M$ is singular with $\mathbf{1}_C$ in its kernel. If F^* solves (3), then for any $t \in \mathbb{R}$ the matrix $F^* + t\mathbf{1}_C e_\ell^\top$ also solves it, because $(L + \mu M)(t\mathbf{1}_C e_\ell^\top) = 0$; and it attains the same objective value, since both the Dirichlet term (the added field is component-wise constant, zero energy) and the fidelity term (no labelled cell in C) are unchanged. Thus the minimiser is non-unique on C . $\square$

Corollary 6 is the formal statement of the limitation the paper foregrounds: smoothing cannot invent a label class that is unreachable from the annotated set. It is exactly the failure mode that the active-sampling experiment attacks by spending its budget in sparse, uncertain regions (Fig. 3c), and it is the mechanism behind the null rare-state recall on the time split, where the rare late-tip cells lie beyond the labelled training distribution (Table 1).

The normalised iteration the code runs. The implementation does not solve (3) directly; it runs the normalised diffusion of Zhou et al. [1]. We prove that this iteration is well posed, converges geometrically, and returns a closed-form fixed point that is itself the unique minimiser of a normalised energy.

Theorem 7 (Spectrum of the normalised adjacency). *Assume every cell has positive degree, $d_i > 0$, and set $S = D^{-1/2}AD^{-1/2}$. Then S is symmetric and every eigenvalue λ of S satisfies $-1 \leq \lambda \leq 1$. Consequently, for any clamping parameter $\alpha \in (0, 1)$, the matrix $I - \alpha S$ is symmetric positive definite and hence invertible, with eigenvalues in $[1 - \alpha, 1 + \alpha] \subset (0, 2)$.*

Proof. Symmetry of S is immediate since $A = A^\top$ and $D^{-1/2}$ is diagonal. For the spectral bound, note S is similar to the random-walk matrix $P = D^{-1}A$ via $S = D^{1/2}PD^{-1/2}$, so S and P share eigenvalues; we bound them on S directly. The normalised Laplacian is $\mathcal{L} = I - S = D^{-1/2}LD^{-1/2}$, which is congruent to $L \succeq 0$ (Lemma 2) through the invertible $D^{-1/2}$, hence $\mathcal{L} \succeq 0$; this gives $S = I - \mathcal{L} \preceq I$, i.e. $\lambda \leq 1$ for every eigenvalue λ of S . For the lower bound, consider $I + S = D^{-1/2}(D + A)D^{-1/2}$, where $D + A$ is the *signless* Laplacian; for any f , $f^\top(D + A)f = \frac{1}{2} \sum_{i,j} A_{ij}(f_i + f_j)^2 \geq 0$ by the same expansion as in Lemma 2 (replace the minus by a plus and use $A_{ij} \geq 0$). Hence $D + A \succeq 0$, so $I + S \succeq 0$ by congruence, giving $\lambda \geq -1$. Therefore every eigenvalue of S lies in $[-1, 1]$. For $\alpha \in (0, 1)$, the eigenvalues of $I - \alpha S$ are $1 - \alpha\lambda \in [1 - \alpha, 1 + \alpha]$, which are strictly positive; since $I - \alpha S$ is symmetric with positive eigenvalues it is positive definite and invertible. $\square$

Theorem 8 (Geometric convergence and closed form of label propagation). *Under the hypotheses of Theorem 7, fix $\alpha \in (0, 1)$, let F_0 be the one-hot seed (labels on annotated cells, zero elsewhere), and run the iteration*

$$F^{(t+1)} = \alpha S F^{(t)} + (1 - \alpha) F_0, \quad F^{(0)} = F_0. \quad (4)$$

Then for every initialisation the iterates converge to the unique fixed point

$$F^* = (1 - \alpha) (I - \alpha S)^{-1} F_0, \quad (5)$$

with the geometric error bound $\|F^{(t)} - F^*\|_F \leq (\alpha)^t \|F^{(0)} - F^*\|_F$. Moreover F^* is the unique minimiser of the normalised graph-regularised energy

$$Q(F) = \frac{1}{2} \text{Tr}(F^\top (I - S)F) + \frac{1-\alpha}{2\alpha} \|F - F_0\|_F^2. \quad (6)$$

Proof. Fixed point. A fixed point F^* of (4) satisfies $F^* = \alpha S F^* + (1-\alpha)F_0$, i.e. $(I - \alpha S)F^* = (1-\alpha)F_0$. By Theorem 7 $I - \alpha S$ is invertible, so the unique fixed point is $F^* = (1-\alpha)(I - \alpha S)^{-1}F_0$, which is (5).

Geometric convergence. Subtract the fixed-point relation from (4):

$$F^{(t+1)} - F^* = \alpha S (F^{(t)} - F^*).$$

Iterating, $F^{(t)} - F^* = (\alpha S)^t (F^{(0)} - F^*)$. Taking Frobenius norms and using submultiplicativity with the spectral norm, $\|F^{(t)} - F^*\|_F \leq \|(\alpha S)^t\|_2 \|F^{(0)} - F^*\|_F = \alpha^t \|S\|_2^t \|F^{(0)} - F^*\|_F$. Since S is symmetric with eigenvalues in $[-1, 1]$ (Theorem 7), $\|S\|_2 = \max |\lambda| \leq 1$, so $\|F^{(t)} - F^*\|_F \leq \alpha^t \|F^{(0)} - F^*\|_F$. As $\alpha \in (0, 1)$, $\alpha^t \rightarrow 0$, giving convergence from any initialisation at the geometric rate α ; the finitely-many (30) iterations the code runs therefore approximate F^* to error $\leq \alpha^{30}$ of the initial gap.

Variational characterisation. Expand Q in (6) using $I - S \succeq 0$ (Theorem 7); it is a sum of two convex quadratics, strictly convex because the second term has Hessian $\frac{1-\alpha}{\alpha} I \succ 0$. Its gradient is

$$\nabla_F Q = (I - S)F + \frac{1-\alpha}{\alpha} (F - F_0).$$

Setting $\nabla_F Q = 0$ and multiplying through by $\frac{\alpha}{1}$ gives $\alpha(I - S)F + (1 - \alpha)(F - F_0) = 0$, i.e. $(\alpha I - \alpha S + (1 - \alpha)I)F = (1 - \alpha)F_0$, which simplifies to $(I - \alpha S)F = (1 - \alpha)F_0$. This is exactly the fixed-point equation, so its unique solution F^* of (5) is the unique minimiser of Q . Hence the deterministic power iteration the code runs converges to the unique minimiser of a normalised Dirichlet-plus-fidelity energy, the normalised counterpart of (2). $\square$

Theorem 8 certifies that the 30-step, $\alpha = 0.9$ diffusion reported in Methods is a faithful, geometrically-convergent surrogate for the closed-form estimator; it is the propagation step whose held-out accuracy and rare-state recall appear in Table 1 and Fig. 3a,b. The active-sampling policy of Fig. 3c then uses the entropy of this same forecast as its uncertainty signal, combined additively (after z -scoring) with the inverse local density that flags the low-density pockets where, by Corollary 6, smoothing alone is uninformative.

1.6 Extended data

Table 2 reports the active-sampling comparison in full, including the rare-query composition that mechanistically explains the rare-recall lift of Fig. 3c, and Table 3 transcribes the run configuration and provenance that fix the benchmark. All entries are transcribed verbatim from `results/summary.json`; no value is recomputed here.

2 Discussion

Within a controlled synthetic benchmark, conditioning single-cell forecasts on the connectivity of the cell k NN graph helps. Label propagation over the graph improves held-out accuracy consistently across four leakage-checked transfer protocols, by roughly four to nine percentage points, and—more usefully—it recovers a rare cell state that an otherwise-matched point classifier almost entirely misses. A topology-aware active-sampling policy compounds the rare-state benefit by directing a fixed annotation budget toward the sparse manifold pockets where rare

Table 2. Extended data: topology-aware versus random active sampling. Rare-state recall on the still-unlabelled cells after spending a fixed query budget of `active_budget`= 60 annotations, and the fraction of queried cells that were genuinely rare (rare-query rate), for the topology-aware policy (uncertainty $\times$ inverse local density) and the uniform-random baseline. Mean $\pm$ 95 % CI over $n = 5$ seeds; the rare base rate is 4 %. The final column is the policy-minus-random difference. Best value per column in **bold**. Transcribed from `results/summary.json` (`active_sampling`).

Policy	Rare-state recall $\uparrow$		Rare-query rate $\uparrow$	
	mean $\pm$	CI	mean $\pm$	CI
Topology-aware	0.3556 $\pm$	0.1597	0.3667 $\pm$	0.0693
Uniform random	0.0247 $\pm$	0.0285	0.0300 $\pm$	0.0160
<i>Difference</i>	0.3309 $\pm$	–	0.3367 $\pm$	–

Table 3. Extended data: benchmark configuration, integrity gates and provenance. Full-scale run parameters, the three integrity checks asserted at build time, and the recorded provenance. Transcribed verbatim from the `config`, `integrity` and `provenance` blocks of `results/summary.json`; the Betti-0 component curve is constant at 1 over $k \in \{2, 4, 8, 16, 32\}$.

Field	Value
Cells / features	3000 / 16
Branches (states incl. rare)	3 (4 states)
Donors / batches / perturbations	8 / 4 / 5
Noise s.d. / rare fraction	0.7 / 0.04
k / label fraction	20 / 0.03
Active budget / seeds	60 / 5
Splits	donor, batch, time, perturbation
Donor split leakage-free	true
Betti-0 monotone in k	true
Rare state present	true
Master seed	0
Python / NumPy / SciPy	3.13.12 / 2.4.3 / 1.17.1
NetworkX / scikit-learn	3.6.1 / 1.8.0
Platform	macOS-arm64 (Mach-0)

cells reside. These results are consistent with the long-standing intuition behind graph-based semi-supervised learning [1, 2] and with the central role of the k NN graph in single-cell analysis [3, 5, 17, 18].

The gains are real but should not be oversold, and two boundaries deserve emphasis. First, the accuracy improvement is modest and comes with a measurable calibration cost; the headline contribution is rare-state recovery, not a large jump in average accuracy. Second, and more fundamentally, the data are synthetic. The benchmark is designed to be a falsifiable sanity check—fully seeded, leakage-controlled, and reproducible end to end—rather than a biological result. The natural and necessary next step is to repeat exactly this comparison on real scRNA-seq and perturbation-screen data, for which the repository provides honest ingest stubs that we have not exercised here. The same protocol could then be strengthened with a learned message-passing propagator [14] in place of fixed-point label propagation, and with calibration-aware training to remove the ECE penalty.

2.1 Limitations

We list the limitations plainly because they bound every claim above. **(1) Synthetic data.** All numbers come from a generated branching lineage, not from biological measurements. We make no biological discovery claim, and the magnitudes reported here need not transfer to real scRNA-seq, where noise, dropout and confounding are far more severe. **(2) Modest accuracy gains.** The accuracy improvement over the baseline is on the order of 0.04 to 0.09 per split (mean 0.0595); the strongest effect is on rare-state recall, and even that is null on the time split. **(3) Calibration cost.** Graph smoothing worsens ECE on all four splits, so the predicted probabilities are less trustworthy as confidences without recalibration. **(4) “Topology” is shallow.** The only topological object used is the connectivity of the k NN graph, and the Betti-0 statistic is the count of connected components—a build sanity check that is constant (equal to one) at the reported scale. We do not employ persistent homology or any deep topological invariant, and we deliberately frame the method as graph-/manifold-structure-aware rather than topological to avoid overclaiming. **(5) Wide intervals on rare metrics.** Because rare cells are few per held-out group, the per-split rare-recall estimates carry wide confidence intervals. **(6) Single benchmark family.** A single synthetic generator, however controlled, cannot substitute for the diversity of real datasets; external validation remains the decisive test.

3 Methods

Synthetic lineage. Cells are generated in an n_{features} -dimensional feature space along a small tree of three branches (a trunk progenitor and two fates that split at pseudotime $t_0 = 0.45$). Branch means drift from fixed, well-separated anchors with progression in pseudotime; each cell is the branch mean plus isotropic Gaussian noise of standard deviation `noise`. Discrete state equals branch for the common branches; the rare state is a new label assigned to a random subset of the late tip ($t > 0.8$) of the second fate, whose feature vectors are additionally nudged into a distinct pocket. Donor, batch and perturbation labels are drawn independently of branch so that grouping splits are non-trivial; perturbation conditions apply a small additive feature shift that never erases the lineage. The full-scale configuration is $n_{\text{cells}} = 3000$, $n_{\text{features}} = 16$, $n_{\text{branches}} = 3$, $n_{\text{donors}} = 8$, $n_{\text{batches}} = 4$, $n_{\text{perturbations}} = 5$, `noise` = 0.7, `rare_fraction` = 0.04, $k = 20$, `label_fraction` = 0.03, `active_budget` = 60, run over $n_{\text{seeds}} = 5$ seeds. Generation is fully seeded and bit-for-bit reproducible.

Cell k NN graph and Betti-0. A symmetric k NN graph is built on cell features by connecting each cell to its k nearest neighbours (self excluded) and symmetrising the directed edges (either-direction convention), exactly as standard single-cell pipelines do before clustering or pseudotime [5]. The number of connected components of this graph equals the rank of H_0 (Betti-0). Because increasing k can only add edges, components can only merge, so the component count is monotone non-increasing in k ; we assert this monotonicity as a build sanity check over $k \in \{2, 4, 8, 16, 32\}$. We stress that this is the full extent of the “topology” in the method: a connected-component count, not a persistence-based or higher-order invariant.

Forecasters. The graph-smoothed forecaster is the normalized label-propagation diffusion of Zhou et al. [1]. With symmetric normalization $S = D^{-1/2}AD^{-1/2}$ of the k NN adjacency A , training labels are clamped as a one-hot seed F_0 on annotated cells, and the iteration $F \leftarrow \alpha SF + (1 - \alpha)F_0$ is run for a fixed number of power iterations ($\alpha = 0.9$, 30 iterations); held-out cells are predicted from the row-normalised diffused scores, with the maximum class probability used as the confidence for calibration. The baseline is a plain k NN classifier in feature space fitted on the same annotated cells with the same k , i.e. identical neighbourhood

information *without* diffusion. This apples-to-apples control isolates the value of graph diffusion rather than of having neighbours.

Splits and annotation. Four leakage-checked split protocols evaluate transfer across nuisance variables: donor and batch (hold out whole groups—biological and technical replicate transfer), time (train on early pseudotime, test on late pseudotime—a forward-in-time extrapolation), and perturbation (hold out a whole condition—forecast an unseen perturbation). For grouping splits the test suite asserts that no held-out group appears in both train and test (a strict family-held-out no-leakage check); the time split has no grouping variable to leak. Only a small fraction (`label_fraction`) of the train pool is annotated, with a light stratification that guarantees a few labels for each present state including the rare one, mirroring how analysts annotate a handful of cells and propagate the rest.

Active sampling. Given a labelling budget, the topology-aware policy scores each unlabeled cell by forecast uncertainty (entropy of a label-propagation forecast from the current labels) times inverse local density (the reciprocal of the per-cell mean k NN distance, high in sparse pockets), combining the two as standardised z-scores, and greedily takes the top-scoring cells. The baseline is uniform random querying. We measure rare-state recall on the still-unlabeled cells after the query and the fraction of queried cells that were rare.

Metrics and aggregation. Accuracy is the fraction of held-out cells classified correctly. Expected calibration error is the population-weighted gap between mean confidence and empirical accuracy over ten confidence bins. Rare-state recall is the fraction of true rare cells recovered (returning 1 by convention when a test set contains no rare cells, so a split is never spuriously penalised). All metrics are averaged over 5 seeds and reported with 95% confidence intervals (1.96 s.e.). All computation is CPU-only (NumPy, SciPy, scikit-learn, NetworkX, Matplotlib); a GPU build is a drop-in but not required.

Experimental design and statistics. Each of the four split protocols is evaluated over the same $n_{\text{seeds}} = 5$ master seeds (seed 0 controls lineage generation, annotation and the random query baseline), giving paired graph-smoothed and k NN measurements per seed and split. Point estimates are seed means; dispersion is the sample standard deviation s with $\text{ddof} = 1$, and the reported uncertainty is the 95% normal-approximation confidence interval $1.96 s/\sqrt{n}$ that supplies every error bar in Figs. 3 and the $\pm$ columns of Tables 1 and 2. We deliberately do not claim statistical significance on the per-split rare-recall estimates: with $\sim 4\%$ rare cells the per-group rare counts are small, the seed-level intervals are correspondingly wide (up to ± 0.3358 on the perturbation split, Table 1), and the appropriate reading is the consistent direction of effect rather than a thresholded p -value. Three integrity gates are asserted before any metric is recorded (Table 3): grouping splits are leakage-free, the rare state is present, and the Betti-0 component count is monotone non-increasing in k (Proposition 4).

Reproducibility. The full-scale run is launched with the command `scripts/run.py` on `configs/full.yaml` writing to `results/`; the file `summary.json` is the single source of truth for every table, figure and numerical claim, and Table 1 is transcribed verbatim from the generated `main_results.tex`. The provenance block records seed, platform, library versions, runtime (20.8 s) and peak memory (320.6 MB).

Data availability

No biological data were used or generated. All results derive from a seeded synthetic lineage that is regenerated deterministically by the code; the run artifacts (`summary.json` and `main_results.md` / `main_results.tex`) and figures are contained in the repository.

Code availability

The reference implementation (`topocell`), configurations, tests and scripts to reproduce every number, table and figure in this manuscript are provided in the repository accompanying this article. Honest ingest stubs for real AnnData and perturbation-screen data are included but are intentionally not exercised here.

Author contributions

M.H. designed the study, implemented the methods and experiments, analysed the results and wrote the manuscript.

Competing interests

The author declares no competing interests.

Acknowledgements

The author received no specific funding for this work and thanks the maintainers of the open-source scientific Python ecosystem (NumPy, SciPy, scikit-learn, NetworkX and Matplotlib) on which the reference implementation is built.

References

- [1] D. Zhou, O. Bousquet, T. N. Lal, J. Weston and B. Schölkopf. Learning with local and global consistency. *Advances in Neural Information Processing Systems (NeurIPS)* **16**, 321–328 (2004).
- [2] X. Zhu and Z. Ghahramani. Learning from labeled and unlabeled data with label propagation. *Technical Report CMU-CALD-02-107*, Carnegie Mellon University (2002).
- [3] L. Haghverdi, M. Büttner, F. A. Wolf, F. Buettner and F. J. Theis. Diffusion pseudotime robustly reconstructs lineage branching. *Nature Methods* **13**, 845–848 (2016).
- [4] C. Trapnell, D. Cacchiarelli, J. Grimsby, P. Pokharel, S. Li, M. Morse, N. J. Lennon, K. J. Livak, T. S. Mikkelsen and J. L. Rinn. The dynamics and regulators of cell fate decisions are revealed by pseudotemporal ordering of single cells. *Nature Biotechnology* **32**, 381–386 (2014).
- [5] F. A. Wolf, P. Angerer and F. J. Theis. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology* **19**, 15 (2018).

- [6] G. La Manno, R. Soldatov, A. Zeisel, E. Braun, H. Hochgerner, V. Petukhov, K. Lidschreiber, M. E. Kastrioti, P. Lönnerberg, A. Furlan *et al.* RNA velocity of single cells. *Nature* **560**, 494–498 (2018).
- [7] E. Becht, L. McInnes, J. Healy, C.-A. Dutertre, I. W. H. Kwok, L. G. Ng, F. Ginhoux and E. W. Newell. Dimensionality reduction for visualizing single-cell data using UMAP. *Nature Biotechnology* **37**, 38–44 (2019).
- [8] L. McInnes, J. Healy and J. Melville. UMAP: Uniform Manifold Approximation and Projection for dimension reduction. *arXiv preprint* arXiv:1802.03426 (2018).
- [9] R. R. Coifman and S. Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis* **21**, 5–30 (2006).
- [10] B. Settles. Active learning literature survey. *Computer Sciences Technical Report 1648*, University of Wisconsin–Madison (2009).
- [11] D. D. Lewis and W. A. Gale. A sequential algorithm for training text classifiers. *Proceedings of the 17th Annual International ACM SIGIR Conference*, 3–12 (1994).
- [12] M. Belkin and P. Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation* **15**, 1373–1396 (2003).
- [13] L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research* **9**, 2579–2605 (2008).
- [14] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)* (2017).
- [15] U. von Luxburg. A tutorial on spectral clustering. *Statistics and Computing* **17**, 395–416 (2007).
- [16] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. On calibration of modern neural networks. *International Conference on Machine Learning (ICML)*, 1321–1330 (2017).
- [17] W. Saelens, R. Cannoodt, H. Todorov and Y. Saeys. A comparison of single-cell trajectory inference methods. *Nature Biotechnology* **37**, 547–554 (2019).
- [18] M. D. Luecken and F. J. Theis. Current best practices in single-cell RNA-seq analysis: a tutorial. *Molecular Systems Biology* **15**, e8746 (2019).

Supplementary Information

A from-scratch derivation of every concept the accompanying code implements. This appendix is written to be read top-to-bottom by someone who has seen linear algebra and basic probability but *not* single-cell biology, graph-based semi-supervised learning, the graph Laplacian, label propagation, calibration or active learning. Every symbol used in the reference implementation (`topocell`) is defined, and each subsection names the source file it maps to, so theory and implementation can be read side by side. The Methods section above gives the condensed version; here we expand it pedagogically without restating the experimental protocol.

S1 The problem in one paragraph

A single-cell experiment measures thousands of individual cells, each described by a vector of features (after dimensionality reduction, a low-dimensional embedding of its gene-expression profile). Cells do not fill this space uniformly: they lie near a low-dimensional curved surface (a “manifold”) that traces differentiation, activation or treatment response. The task we study is *forecasting*: predict the discrete state of a cell under a condition we did not directly observe—a donor not yet profiled, a later technical batch, a later point along the trajectory, or an untested perturbation. Two facts make this hard. (i) Only a handful of cells are ever annotated by hand; the rest are unlabelled. (ii) The biologically decisive states (transient progenitors, treatment-resistant clones) are often *rare*, and ordinary classifiers trained to minimise average error ignore them. The scientific question of this paper, kept deliberately narrow and falsifiable: does conditioning a forecaster on the *connectivity* of the cell k -nearest-neighbour (k NN) graph—rather than on point features alone—improve (a) generalisation to held-out groups and (b) recovery of a rare state? The answer is yes, but modestly on accuracy, more clearly on rare-state recall, and at a small calibration cost; all data are synthetic.

S2 The data: a seeded synthetic branching lineage

Source: `src/topocell/synthetic.py`. We do not ship or fabricate real biological data (see `ingest.py` for honest hooks into real `AnnData`). Instead we *generate* a fully controlled lineage with the same structure the method must handle, so every claim is checkable bit-for-bit.

A “lineage” is a tree of branches over a continuous coordinate $t \in [0, 1]$ called *pseudotime* (how far a cell has progressed). The tree here has three branches (`_TREE` in `synthetic.py`): branch 0 is the trunk/common progenitor, living over $t \in [0, 1]$; branch 1 is the first fate, which splits off the trunk at $t_0 = 0.45$; and branch 2 is the second fate, which also splits at $t_0 = 0.45$. Each branch has a fixed, well-separated *anchor* point in feature space. A cell on branch b at pseudotime t has a mean feature vector $\mu(b, t)$ that drifts linearly from the split point toward that branch’s anchor (function `_branch_mean`). The cell itself is

$$x = \mu(b, t) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \text{noise}^2 I).$$

The key geometry: early cells (small t) of different fates share the same trunk mean, so they *overlap* and are hard to separate, whereas late cells are well resolved. This is exactly the regime in which letting a label diffuse along the manifold should beat a point classifier.

Each cell also carries the metadata a real forecaster would use: **state** (the discrete label to predict, equal to the branch for the common branches); **donor** (a biological-replicate id); **batch** (a technical-replicate id); **perturbation** (a condition id, with 0 the control, applying a small additive feature shift that never erases the lineage); and **pseudotime** (the continuous progression coordinate). Donor, batch and perturbation are drawn *independently* of the biological branch, so the held-out splits of Section S7 are non-trivial (a donor spans multiple branches).

The rare state. A brand-new state label ($\text{id} = n_{\text{branches}}$) is carved out of the late tip ($t > 0.8$) of the second fate: a random `rare_fraction` of cells there are relabelled and additionally *nudged* into their own pocket of feature space ($X[\text{chosen}] += [1.5, 1.0, 0, \dots]$). With `rare_fraction` = 0.04 this is $\sim 4\%$ of cells—the transient subpopulation an analyst most wants to find. Everything is seeded with NumPy’s `default_rng(seed)`, so the lineage is bit-for-bit reproducible across machines.

S3 The graph object: the k NN cell graph

Source: `src/topocell/graph.py`. The bridge from “points in feature space” to “topology” is the k -nearest-neighbour graph, the same construction SCANPY uses before clustering or pseudotime [5]. To build it (`build_knn_graph`): for each cell, find its k nearest cells by Euclidean distance (self excluded), giving a *directed* k NN (an edge $i \rightarrow j$ if j is among i ’s k neighbours); then *symmetrise* with the either-direction convention $A \leftarrow \max(A, A^\top)$, so an undirected edge $\{i, j\}$ exists if i picked j or j picked i . Symmetrising is the standard single-cell convention and makes connected components well defined. The result is a symmetric adjacency matrix A ($A = A^\top$, $A_{ii} = 0$). A is the graph, and it is the *substrate* for everything that follows: labels diffuse over its edges, and rare cells are hunted in its sparse regions. The *degree* of cell i is $d_i = \sum_j A_{ij}$ (how many neighbours it has); collect the degrees on the diagonal of $D = \text{diag}(d_1, \dots, d_n)$.

S4 Topology: Betti-0 as a connectivity diagnostic

Source: `graph.py`. The only topological invariant the method uses is Betti-0, the number of connected components of the graph, equal to the rank of the 0-th homology group H_0 (`n_connected_components` computes it with SciPy’s `connected_components`). Intuitively, a connected component is a maximal set of cells reachable from one another by walking along edges; the number of them counts how many disjoint pieces the data splits into. A real, testable property: as k grows, edges are only *added*, so components can only *merge*—the component count is monotone non-increasing in k . `betti0_curve` returns this count for $k \in \{2, 4, 8, 16, 32\}$, and `tests/test_betti_monotonic.py` asserts the monotonicity as a build sanity check. Honesty note: at the reported scale the graph is a single connected component for every k in that set (the Betti-0 curve is constant at 1), so Betti-0 here is a *passive* sanity check, not an informative descriptor—which is precisely why the paper frames the method as graph-/manifold-structure-aware, not as deep topology (no persistent homology, no higher-order invariants). `topology_features` and `local_density` add cheap graph-level descriptors (mean degree, clustering, and a per-cell inverse-density proxy) used only by the active-sampling policy of Section S8.

S5 The graph Laplacian and Dirichlet energy

This is the heart of the mathematics. Define the (combinatorial) graph Laplacian $L = D - A$. Its single most important property is that, for any vector f on the cells,

$$f^\top L f = \frac{1}{2} \sum_{i,j} A_{ij} (f_i - f_j)^2 \geq 0.$$

This is the *Dirichlet energy*: it is small exactly when f varies smoothly across edges (neighbouring cells get similar values) and large when f jumps across edges. L is symmetric positive semidefinite; the all-ones vector lies in its null space (constant labels cost zero energy), and the multiplicity of the eigenvalue 0 equals the number of connected components (tying back

to Betti-0). “Smoothing labels over the graph” will mean choosing f that is smooth in this Dirichlet sense while still matching the few labels we have, made precise next.

S6 Label propagation as a minimum-Dirichlet-energy label extension

Source: `src/topocell/policy.py`. We observe one-hot labels Y on a small labelled set (mask M with $M_{ii} = 1$ if cell i is labelled, else 0) and want a soft-label matrix F (rows are cells, columns are classes). The variational objective—stated as Theorem 5 above—is

$$J(F) = \text{Tr}(F^\top L F) + \mu \|M(F - Y)\|_F^2.$$

The first term is the Dirichlet energy of Section S5 (smoothness on the graph); the second is a *fidelity* penalty (stay near the observed labels on labelled cells), with $\mu > 0$ the trade-off. J is convex; setting the gradient to zero gives

$$2LF + 2\mu M(F - Y) = 0 \implies (L + \mu M)F = \mu MY.$$

On any connected component containing at least one labelled cell, $L + \mu M$ is positive definite, so the minimiser is *unique*: label propagation is the unique minimum-Dirichlet-energy extension of the observed labels. The same theorem states the limitation—a component with no labels is not identified by smoothing alone—which is exactly the failure mode that active sampling (Section S8) attacks.

The actual iteration. The implementation runs the normalised form of Zhou et al. [1]. Symmetrically normalise the adjacency, $S = D^{-1/2}AD^{-1/2}$ (eigenvalues in $[-1, 1]$); seed F_0 as one-hot labels on labelled cells and zero elsewhere; and iterate

$$F \leftarrow \alpha(SF) + (1 - \alpha)F_0.$$

This is a damped power iteration whose fixed point $F^* = (1 - \alpha)(I - \alpha S)^{-1}F_0$ is the closed-form solution of a normalised version of $J(F)$, with α the diffusion/clamping trade-off. The code uses $\alpha = 0.9$ and 30 power iterations (deterministic, CPU, no learning). Held-out cells are predicted by the argmax of their row-normalised diffused scores; the maximum class probability is the *confidence* used for calibration (Section S9). This fixed-point solver is a faithful surrogate for a message-passing GNN: dropping in a learned propagator [14] leaves the rest of the pipeline unchanged.

The honest baseline. The `KNNForecaster` is a plain k -nearest-neighbour classifier in feature space, fitted on the *same* labelled cells with the *same* k , but with *no* diffusion. It sees identical neighbourhood information; the only difference is that it does not let labels spread along the graph. Comparing `LabelPropagation` to this baseline isolates the value of graph diffusion, not the mere availability of neighbours—the apples-to-apples control that is the methodological spine of the paper.

S7 Splits, leakage and the few-label regime

Source: `src/topocell/splits.py`. A forecaster that only works when train and test share donors, batches, conditions or timepoints is a *memoriser*, not a forecaster. So we evaluate transfer across nuisance variables under four protocols (`SPLIT_TYPES`): **donor** (hold out whole donors—biological-replicate transfer); **batch** (hold out whole batches—technical-replicate transfer); **time** (train on early pseudotime $t < \text{cutoff}$, test on late—a genuine forward-in-time extrapolation); and **perturbation** (hold out a whole condition—forecast an unseen perturbation).

For grouping splits, `has_leakage()` checks that the held-out group’s id appears in test but not in train (disjoint group sets), and the test suite asserts this strict family-held-out no-leakage property. The time split cuts on a continuous coordinate, so there is no grouping variable to leak (clean by construction).

Few-label annotation. Only a fraction `label_fraction` of the train pool is actually annotated (`_annotate`), mirroring how analysts label a handful of cells and propagate the rest. The selection is lightly *stratified*: a couple of labels are guaranteed for each present state, including the rare one—with zero rare labels no method could recall the rare state and the comparison would be uninformative rather than honest. The held-out test cells are never annotated.

S8 Active sampling: hunting the rare state

Sources: `policy.py`, `graph.py`. Active learning chooses *which* cells to annotate next, given a fixed labelling budget. The topology-aware policy (`active_sample`, `strategy='topology'`) scores each unlabelled cell by

$$\text{score} = \text{zscore}(\text{uncertainty}) + \text{zscore}(\text{inverse local density}),$$

and greedily takes the top `budget` cells. The two ingredients are: (i) *uncertainty*, the Shannon entropy $H(p) = -\sum_c p_c \log p_c$ of a label-propagation forecast from the *current* labels (`_entropy`), where high entropy means the model is unsure—near a decision boundary, the classic active-learning signal; and (ii) *inverse local density*, the reciprocal of a cell’s mean k NN distance (`graph.local_density`), since rare states sit in low-density pockets, making $1/\text{density}$ a cheap, topology-aware “where are the rare cells” signal. Standardising each to a z-score (subtract mean, divide by standard deviation) puts the two heterogeneous signals on a common scale before adding them. The baseline is uniform *random* querying (`strategy='random'`). `recall_after_query` measures rare-state recall on the still-unlabelled cells before versus after adding the queried set; the gap is the active-sampling *lift*. We also report the fraction of queried cells that were genuinely rare (`rare_query_rate`).

S9 Metrics and aggregation

Source: `src/topocell/metrics.py`. *Accuracy* is the fraction of held-out cells classified correctly. *Expected calibration error* (ECE) asks whether the confidences are trustworthy: bin the test cells by predicted confidence into ten bins, in each bin compare mean confidence to empirical accuracy, and take the population-weighted average gap; a perfectly calibrated forecaster has $\text{ECE} \rightarrow 0$. We report ECE precisely because graph smoothing sharpens probabilities and, in this setting, makes them mildly overconfident—a small, honestly disclosed cost that recalibration in the manner of Guo et al. [16] would remove. *Rare-state recall* is the fraction of true rare cells the model labels rare; by convention it returns 1 when a test set has no rare cells (nothing to miss), so a split is never spuriously penalised. *Aggregation* (`summarize`): every metric is computed per seed and reported as mean, sample standard deviation s (with $\text{ddof} = 1$) and a 95% confidence interval $1.96 s / \sqrt{n}$ over the $n = 5$ seeds. These `ci95` values are exactly the error bars in the figures and the $\pm$ in the text.

S10 Reproducibility object: one source of truth

Sources: `runner.py`, `scripts/`. `runner.py` ties everything together: generate the lineage, build the graph, check integrity (leakage-clean splits, rare state present, Betti-0 monotone), run both

forecasters under all four splits over five seeds, run the active-sampling comparison, and write `results/summary.json`—the single source of truth. Nothing in the paper is typed by hand:

```
summary.json  $\xrightarrow{\text{make\_tables.py}}$  results/main_results.tex,
summary.json  $\xrightarrow{\text{make\_figures.py}}$  figures/fig_{schematic,lineage,forecast}.pdf.
```

`scripts/audit_claims.py` is a readiness gate: it checks that every headline number is present and traceable, that the integrity flags are true, and (against the repository README) that the mandated reproducibility phrase is present with no forbidden superiority claims; it exits non-zero if any check fails. Figures are drawn by `src/topocell/plotting.py` in *Nature Machine Intelligence* house style (vector PDF with embedded fonts, sans-serif, colour-blind-safe Okabe–Ito categories with viridis for the continuous embedding axis, bold lower-case panel labels, 95% CI error bars, no in-plot titles); `fig_schematic` is the method-overview Figure 1.

To reproduce everything from the `code/` folder:

```
export KMP_DUPLICATE_LIB_OK=TRUE
pip install -e . # or: pip install -e .[gpu]
bash scripts/reproduce_all.sh full # writes summary.json, tables, figures
```

The reported run uses 3000 cells, 16 features, 8 donors, 4 batches, 5 perturbations, 3 branches, ~4% rare state, $k = 20$, `label_fraction = 0.03`, `active_budget = 60`, and 5 seeds with master seed 0; the provenance block records a runtime of 20.8s and peak memory of 320.6 MB on a laptop CPU. The figure and table artifacts are copied verbatim into the `submission/` folder used to build this manuscript.

S11 What the result means (and what it does not)

Within this controlled synthetic benchmark, conditioning forecasts on k NN-graph connectivity helps: label propagation beats the matched point classifier on held-out accuracy on all four splits (mean gain ≈ 0.0595 , roughly four to nine percentage points) and—the clearer effect—recovers a rare state the baseline almost never finds. The topology-aware active-sampling policy raises rare-state recall from 0.0247 (random) to 0.3556. These gains carry a small, consistent calibration penalty (ECE is slightly worse on every split). The accuracy gain is real but modest; the headline contribution is rare-state recovery, and even that is null on the time split (forward-in-time extrapolation puts the rare late-tip cells entirely outside the training distribution—reported as-is). The critical caveats, expanded in the Limitations above, are that all data are synthetic (no biological discovery claim); that the “topology” is shallow (k NN-graph connectivity and a Betti-0 component count that is constant at this scale, not persistent homology); that rare-metric confidence intervals are wide because rare cells are few; and that a single synthetic generator cannot substitute for real datasets. The decisive next step is to repeat this exact comparison on real scRNA-seq and perturbation-screen data via the unexercised ingest stubs.